

第四章作业答案

1

给定字符串 S （长度 n ）和 T （长度 m ），请返回 S 中能覆盖 T 所有字符（每种字符出现次数不少于 T 中对应次数）的最短子串及其起始下标（从 0 开始）；若不存在，返回 $("", -1)$ 。若 T 为空，返回 $("", 0)$ 。

例如 $S = \text{"ADOBECODEBANC"}$ ， $T = \text{"ABC"}$ ，返回 $(\text{"BANC"}, 9)$ 。

要求：时间复杂度 $O(n + m)$ （或 $O(n)$ ，系数合理即可），可借助 $O(|\Sigma|)$ 额外空间。请说明思路并给出伪代码。

解答：

思路。用两个计数表 $need$ 与 $window$ 分别记录 T 中各字符所需次数与当前窗口内各字符出现次数。维护 $have$ （窗口内已达标的字符种类数）和 $required = |need|$ 。右指针扩张窗口直到 $have == required$ ，再不断左移左指针尝试缩小窗口。只记录最佳窗口的起点和长度，最后截取一次子串。

伪代码。

```
function minWindow(S, T):
    if len(T) == 0: return ("", 0)
    if len(S) < len(T): return ("", -1)

    need = {}
    for c in T:
        need[c] = need.get(c, 0) + 1

    window = {}
    have = 0
    required = len(need)
    left = 0
    best_start = -1
    min_len = +∞

    for right = 0 .. len(S)-1:
        c = S[right]
        if c in need:
            window[c] = window.get(c, 0) + 1
            if window[c] == need[c]:
                have += 1

        while have == required and left <= right:
            cur_len = right - left + 1
            if cur_len < min_len:
                min_len = cur_len
                best_start = left
```

```

        // 尝试左移
        c2 = S[left]
        if c2 in need:
            window[c2] -= 1
            if window[c2] < need[c2]:
                have -= 1
        left += 1

    if best_start == -1: return ("", -1)
    return (S[best_start .. best_start + min_len - 1], best_start)

```

复杂度。两个指针各至多移动 n 次，建立 `need` 需 $O(m)$ ，最后一次截取需至多 $O(n)$ ；总时间 $O(n + m)$ ，额外计数空间 $O(|\Sigma|)$ （不计返回的子串）。

2

给定模式串 P （长度 m ）和文本串 T （长度 n ）， P 与 T 均只含普通字符（无通配符）。请计算并返回 P 在 T 中所有出现的位置的起始下标（从 0 开始，允许重叠）。若 P 为空，返回空列表。

例如 $P = \text{"abab"}$ ， $T = \text{"ababab"}$ ，返回 $[0, 2]$ 。

要求：时间复杂度 $O(n + m)$ ，除结果列表外的额外空间 $O(m)$ （用于课件定义的 `next` 数组）。请说明思路并给出伪代码。

解答：

思路。按课件的约定，`next[0] = -1`；对 $i > 0$ ，`next[i]` 是 $P[0..i-1]$ 的最长相等真前后缀长度。为处理完整匹配，沿用同一定义额外计算 `next[m]`，数组长度为 $m + 1$ 。文本指针 i 指向待比较字符， j 是当前已匹配的模式前缀长度。失配时令 $j = \text{next}[j]$ ，文本指针不回退；若 $j == -1$ ，则直接读入下一个文本字符并把 j 置回 0。当 $j == m$ 时，记录 $i - m$ ，再令 $j = \text{next}[m]$ ，便可继续查找重叠出现。

伪代码。

```

function getNext(P):
    m = len(P)
    next = [-1] * (m + 1)      // next[0] = -1; 多算一项 next[m]
    i = 0
    k = -1
    while i < m:
        if k == -1 or P[i] == P[k]:
            i += 1
            k += 1
            next[i] = k
        else:
            k = next[k]
    return next

function kmpSearch(P, T):
    m, n = len(P), len(T)

```

```

    if m == 0: return []
    next = getNext(P)
    result = []
    i = 0
    j = 0
    while i < n:
        if j == -1 or T[i] == P[j]:
            i += 1
            j += 1
            if j == m:
                result.append(i - m)
                j = next[m]                // 保留可重叠的已匹配前缀
        else:
            j = next[j]
    return result

```

复杂度。 时间 $O(n + m)$ ；除结果列表外的额外空间 $O(m)$ 。

3

给定字符串 s （长度 n ），请利用 KMP `next` 数组计算 s 的**最短周期**。若 s 为空，返回 0。

若 s 可由某个长度 $p < n$ 的子串重复多次构成（即 $s = (x)^k$ 且 $|x| = p$ ），则称 p 为 s 的一个周期，否则记最短周期为 n （即 s 本身）。

例如：

- $s = \text{"abccabccabcc"}$ → 最短周期 3（"abc" 重复 3 次）
- $s = \text{"abccab"}$ → 最短周期 5（无非平凡周期）
- $s = \text{"aaaaa"}$ → 最短周期 1

要求：时间复杂度 $O(n)$ ，额外空间 $O(n)$ （用于 `next` 数组）。请说明思路并给出伪代码。

解答：

思路。 用上题的 `getNext(S)`，其中额外计算的 `next[n]` 正是整个 s 的最长相等真前后缀长度，记为 L 。令 $p = n - L$ 。若 $p < n$ 且 $n \% p == 0$ ，则最短周期为 p ；否则为 n 。

直观理解。 $s[0..n-p-1] = s[p..n-1]$ 恰好是 border 的语义（前后缀相等，长度 $n - p = L$ ）；再加上 $n \% p == 0$ 这一条件，就能推出 s 是 $(s[0..p-1])^k$ 。

伪代码。

```

function shortestPeriod(S):
    n = len(S)
    if n == 0: return 0
    next = getNext(S)           // 复用上题的 getNext
    L = next[n]                 // 整个 S 的最长 border 长度
    p = n - L
    if p < n and n % p == 0:
        return p
    else:
        return n

```

复杂度。 `getNext` 为 $O(n)$ ，主函数常数时间；总计 $O(n)$ 时间、 $O(n)$ 额外空间（`next` 数组）。

4

给定字符串 `s`（长度 `n`，以字符数组形式存储，可就地修改）。

在一次从左到右扫描中，**原地**（ $O(1)$ 额外空间）删除 `s` 中所有字符 `'b'`，并删除所有出现的子串 `"ac"`（删除要级联）。要求**单次遍历**，最终把保留的字符移到 `s` 的左端，并返回新长度。

请给出：**思路说明、伪代码、以及时间复杂度分析。**

解答：

思路。 使用两个指针 `i`（读指针）和 `j`（写指针）：

- 初始 `i = 0`，`j = 0`。
- 对每个 `s[i]`：
 1. 若 `s[i] == 'b'`，跳过（仅 `i++`，不写入）。
 2. 否则待写入字符为 `c = s[i]`。若 `c == 'c'` 且 `j > 0` 且 `s[j-1] == 'a'`（`s[j-1]` 是写入缓冲区的最后一个有效字符），则说明形成一个待删除的 `"ac"`，应当把那个 `'a'` 弹出（`j--`）并跳过当前 `'c'`（仅 `i++`，不写入）。
 3. 否则（不属于上述两种情形），将 `s[j] = c`，`j++`，`i++`。

由于我们把弹出的 `'a'` 也即写缓冲区中可能再次出现的尾字符与后续字符的匹配考虑进来了，整个处理是**线性、单次遍历**的；级联删除通过 `j` 减小并在新形成的尾字符上继续比较实现。

伪代码。

```

function remove_b_and_ac_inplace(s):
    n = len(s)
    i = 0
    j = 0
    while i < n:
        c = s[i]
        if c == 'b':
            i += 1
            continue

```

```
    if c == 'c' and j > 0 and s[j - 1] == 'a':
        j -= 1                                // 弹出 'a', 跳过当前 'c'
        i += 1
        continue
    s[j] = c
    j += 1
    i += 1
return j                                    // 新长度
```

复杂度分析。

- 每个字符被读取一次；写、读、比较均为 $O(1)$ 。指针 i 、 j 单调右移， j 仅在弹出 'a' 时减小，但整个过程中 $i + j$ 的总增量不超过 $2n$ ，故总时间 $O(n)$ 。
- 额外空间 $O(1)$ （只用到几个指针 / 临时变量，且 s 被就地修改）。